Quantization-Aware Differentiable Architecture Search for Photonic-Compatible Neural Networks

Filippo Lucchesi

***Abstract*—Photonic neural networks promise high-throughput, low-latency inference, but their limited numerical precision and physical constraints are rarely considered during neural architecture search. This article presents QA-DARTS, which makes quantization part of differentiable architecture search rather than a post-search operation. Learned-step-size quantization is applied to weights and activations while architecture parameters are optimized, so candidate operations are compared under the precision at which the selected network will operate. The photonic-compatible search space combines residual convolutional blocks, efficient and dilated operators, spatial self-attention, and alternative downsampling paths. Precision-specific training schedules support six-, four-, and two-bit search, followed by deterministic reconstruction and retraining from scratch. The evaluation covers CIFAR-10 and a classification adaptation of the German Traffic Sign Detection Benchmark.**

***Index Terms*—Differentiable architecture search, hardware-aware neural architecture search, learned step size quantization, neural architecture search, photonic neural networks, quantization-aware training.**

I. Introduction

DEEP neural networks increasingly rely on accelerators to meet the latency and energy requirements of real-time inference. Photonic neural networks (PNNs) are a promising accelerator family because optical propagation naturally supports high bandwidth, parallel signal transport, and very low propagation latency. Integrated photonic circuits can implement matrix–vector multiplication through interferometric meshes, wavelength-selective weight banks, or hybrid photonic–electronic multiply–accumulate units [1], [3]. These properties make photonics attractive for workloads in which dense linear transformations dominate the execution time.

The same physical mechanisms that make PNNs attractive also impose non-idealities that are not represented by an ordinary floating-point neural network. Optical sources, modulators, detectors, and data converters introduce noise and restrict numerical resolution. Inputs encoded as optical intensity are non-negative. Optical fan-in is limited by power splitting, crosstalk, and component count, while large spatial kernels require additional hardware and optical routing. A network that performs well in an unconstrained software environment can therefore be expensive, inaccurate, or impossible to map to a photonic platform. Photonic-aware neural networks address part of this gap by incorporating device-level constraints into architecture and training choices [2], but those choices are commonly made by hand.

F. Lucchesi is with the Department of Information Engineering, M.Sc. Program in Artificial Intelligence and Data Engineering. Contact information will be added before submission.

Neural architecture search (NAS) offers a systematic alternative. In particular, differentiable architecture search (DARTS) replaces discrete architectural decisions with continuous mixtures and optimizes their mixing coefficients by gradient descent [5]. Its computational efficiency makes DARTS suitable for co-design problems in which many candidate networks must be evaluated. This work extends that formulation so that low-precision behavior is present during search; the motivation and distinction from post-search quantization are developed in Section III.

The current implementation extends the original formulation substantially. Its contributions are as follows.

- It provides quantization-aware NNI mixed operations for convolution, linear layers, and multi-head self-attention. Convolutional weights use per-output-channel Learned Step Size Quantization (LSQ), while linear and attention projections use per-tensor LSQ.
- It defines a seven-stage, always-residual search space that contains 13 independent decisions and 13,824 discrete architectures. In addition to standard and depthwise-separable convolutions, the space includes dilated depthwise-separable operators, zero main branches that realize residual-only blocks, quantized spatial self-attention, and two downsampling mechanisms.
- It replaces the former hand-selected residual coefficient with a learned scale constrained to the interval $(0, 2)$ and quantizes the main and residual branches separately before addition. This makes residual fusion part of the learned low-precision model.
- It introduces precision-dependent stabilization. The two-bit CIFAR-10 search starts with a full-precision warm-up and a widened backbone, whereas the two-bit traffic-sign search follows a $6 \rightarrow 4 \rightarrow$ 2-bit curriculum with quantizer-state transitions and an architecture-parameter warm-up.
- It separates search, discretization, retraining, and evaluation. The selected architecture is reconstructed as a deterministic model and trained from random initialization, preventing inherited supernet weights from confounding the final assessment.

The remainder reviews the background, motivates QA-DARTS before describing its NNI implementation, and defines the experimental protocol. Results are reserved for the current experiments and plots.

II. Background and Related Work

A. Photonic Neural Computing

PNNs implement neural computations using optical signals and integrated photonic components. A coherent architecture can encode a matrix through a mesh of Mach–Zehnder interferometers, while wavelength-division multiplexed systems can assign weights to optical carriers using microring resonators or semiconductor optical amplifiers [1]. Hybrid designs often retain nonlinear activation, storage, and control in electronics while executing the most expensive linear operations optically. The precise system architecture changes the distribution of optical and electronic cost, but four constraints are consistently relevant to neural-network design.

First, numerical precision is limited. The effective number of bits is determined jointly by analog noise, device variability, thermal drift, optical loss, and data-converter resolution. This work considers six bits as the upper target and studies four- and two-bit operation as increasingly aggressive regimes. Second, optical intensity is non-negative, so activation paths are designed around non-negative tensors and ReLU-like nonlinearities. Signed quantities are still required for weights, normalized residual projections, and attention scores; these tensors receive explicit signed quantizers. Third, the fan-in of a photonic neuron cannot grow without bound. The classification head is therefore preceded by global average pooling and receives exactly 200 features. Fourth, experimentally practical spatial kernels are small. Every spatial convolution in the proposed backbone uses a 3×3 kernel or a 3×3 dilated kernel; pointwise projections use 1×1 kernels.

Photonic awareness is not equivalent to quantization alone. A model can satisfy a bit-width constraint yet use an unsuitable fan-in, negative optical input, excessive channel expansion, or an operation whose conversion cost overwhelms its arithmetic benefit. Hardware-aware search must therefore represent both numerical and structural constraints rather than treating bit width as a complete proxy for deployability.

B. Neural Architecture Search and DARTS

NAS methods automate decisions that would otherwise require extensive manual exploration [4]. Early approaches trained many independent architectures, which made the search prohibitively expensive for all but the largest compute budgets. Weight-sharing one-shot methods reduce that cost by embedding many candidates in a common supernet [7], [8]. DARTS goes further by expressing each discrete choice as a differentiable mixture.

Let $\mathcal{O} = \{o_1, \ldots, o_K\}$ be the candidates at one decision point and let $\boldsymbol{\alpha} \in \mathbb{R}^K$ be its architecture logits. The relaxed operation is

$$\bar{o}(x) = \sum_{k=1}^{K} p_k o_k(x), \qquad p_k = \frac{\exp(\alpha_k)}{\sum_{j=1}^{K} \exp(\alpha_j)}. \tag{1}$$

Network parameters w and architecture parameters α are learned through the bilevel problem

$$\min_{\alpha} \ \mathcal{L}_{\mathrm{val}}(w^\star(\alpha), \alpha) \quad \text{s.t.} \quad w^\star(\alpha) = \arg\min_{w} \mathcal{L}_{\mathrm{train}}(w, \alpha). \tag{2}$$

In practice, the two parameter groups are updated alternately using mini-batches from the training and validation partitions. At the end of search, each mixture is discretized with $k^\star = \arg\max_k \alpha_k$.

This relaxation introduces known biases. Parameter-free paths can learn faster than convolutional paths, mixed operations can exploit the simultaneous capacity of several candidates, and weakly polarized probabilities can make the final argmax unstable. Consequently, the relaxed supernet may perform well even when the final argmax decision is poorly separated from its alternatives. Entropy and probability margins are useful diagnostics of whether discretization represents a confident architectural choice.

C. Quantization-Aware Training

Quantization maps a continuous tensor to a finite set of representable values. A uniform signed quantizer with step size $s > 0$, lower integer bound Q_n, and upper bound Q_p can be written as

$$q(x; s) = s \,\mathrm{clip}\Big(\mathrm{round}\Big(\frac{x}{s}\Big), Q_n, Q_p\Big). \tag{3}$$

For a b-bit signed tensor, the usual integer bounds are $Q_n = -2^{b-1}$ and $Q_p = 2^{b-1} - 1$; an unsigned tensor instead uses $Q_n = 0$ and $Q_p = 2^b - 1$. Rounding has zero derivative almost everywhere, so quantization-aware training commonly uses the straight-through estimator (STE), treating the derivative through the rounding step as the identity inside the clipping range [11], [12].

DoReFa-Net demonstrated effective low-bit training through fixed transformations of weights and activations [9]. Learned Step Size Quantization (LSQ) instead learns s jointly with the network and scales its gradient to prevent the step-size update from dominating optimization [10]. With N tensor elements, this scale is proportional to

$$g_s = \frac{1}{\sqrt{N Q_p}}. \tag{4}$$

An activation quantizer may also learn an upper clipping parameter. This parameter plays a role analogous to parameterized clipping in PACT [13]: it balances saturation error against the resolution lost when a fixed number of levels spans an unnecessarily wide interval.

D. Efficient and Context-Aware Operators

Standard convolution jointly learns spatial and channel mixing. A depthwise-separable block factorizes that computation into a channel-wise convolution and a pointwise projection, reducing parameter and multiply–accumulate counts as in MobileNet [14]. Dilated convolution increases the receptive field without enlarging the physical kernel [15]. These operators offer different balances between capacity, fan-in, and implementation cost.

Multi-head self-attention provides a complementary global, content-dependent interaction mechanism [16]. For a sequence of N spatial tokens its score matrix scales quadratically with N, making attention more practical at low-resolution stages. Its normalization, signed scores, bounded probabilities, and

multiple linear projections also create quantization behavior that differs substantially from convolution.

III. From DARTS to Quantization-Aware DARTS

A. The Role of Quantization

Quantization reduces the precision used to represent and process weights and activations. Besides lowering memory and digital arithmetic costs, it is especially relevant to photonic accelerators because their effective resolution is bounded by analog noise, loss, device variation, and data conversion. It is therefore not merely a compression step applied after network design: at low precision it changes the numerical behavior of the operators from which an architecture is assembled.

Conventional quantization-aware training (QAT) retains latent full-precision weights for gradient-based optimization but uses quantized copies in the forward pass. An input quantizer first maps the incoming activation to a finite codebook, and a weight quantizer does the same for the layer parameters before the operation is evaluated. The STE supplies approximate gradients through rounding, allowing the latent weights and quantizer parameters to adapt to clipping and quantization error. Figure 1(a) will summarize this single-layer mechanism.

When QAT follows a conventional DARTS search, however, the two stages solve different problems. DARTS ranks mixed operations using full-precision tensors, selects one operation at every decision, and only then adapts the discrete network to low precision. The selected architecture is thus optimal only with respect to the full-precision search surrogate. Quantization may alter the ranking because candidate families differ in dynamic range, parameter distribution, accumulation depth, normalization, and sensitivity to clipping. Post-search QAT can adapt the selected weights, but it cannot revisit an architectural decision made without those effects.

B. Quantization Inside the Differentiable Search

QA-DARTS removes this mismatch by placing QAT inside each candidate of the relaxed supernet. Let o_k be a candidate with latent weights W_k, let $Q_{w,k}$ denote its weight quantizer, and let Q_x and $Q_{a,k}$ denote activation quantizers at the relevant input and output boundaries. The mixed operation becomes

$$\bar{o}_q(x) = \sum_{k=1}^{K} p_k \, Q_{a,k}(o_k(Q_x(x); Q_{w,k}(W_k))) \,, \qquad (5)$$

where the probabilities p_k are still obtained from the architecture logits as in (1). The target bit width is a fixed condition of an experiment, not another candidate in the architecture space. Each operation retains its own latent weights and weight quantizer, while activation quantizers are shared only where candidates genuinely consume the same tensor and require the same numerical interpretation.

The decisive change is the signal used to update the architecture logits. Because both the training update of W_k and the validation update of α_k pass through the quantized supernet, the probabilities respond to accuracy under clipping, rounding, and restricted dynamic range. The search can therefore prefer an operation that is slightly weaker in floating point but more stable at the deployment precision. Figure 1(b) will contrast this joint process with the sequential DARTS-then-QAT pipeline.

QA-DARTS does not claim that quantization alone captures an entire photonic platform. Its purpose is to make numerical precision part of architecture selection while the model space separately enforces structural constraints such as kernel size, fan-in, tensor non-negativity at optical interfaces, and feasible downsampling. After search, the maximum-probability choices are reconstructed as one deterministic network and retrained from random initialization under the same quantization policy. This separates the quality of the selected architecture from the inherited state of the shared supernet.

IV. NNI Implementation of QA-DARTS

A. System Overview

The implementation is built on the ModelSpace, LayerChoice, and DARTS components of Microsoft Neural Network Intelligence (NNI) [6]. NNI mutates ordinary mutable layers into mixed operations before one-shot training begins. QA-DARTS supplies custom mutation hooks for mutable convolution, linear, and multi-head-attention modules. Each hook constructs the corresponding quantized mixed operation, transfers the supernet weights, attaches the appropriate LSQ modules, and preserves the fixed target precision as metadata rather than exposing it as an architecture choice.

The end-to-end workflow is summarized below:

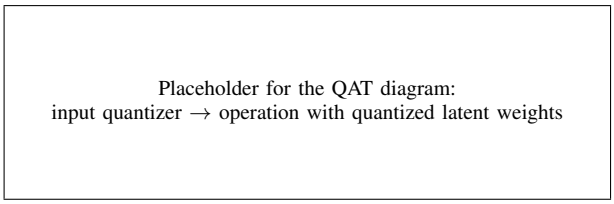
$$\text{constrained model space} \rightarrow \text{NNI mutation}$$
$$\rightarrow \text{quantized supernet search} \rightarrow \text{argmax export}$$
$$\rightarrow \text{deterministic reconstruction} \rightarrow \text{training from scratch.}$$

The search and deterministic models implement the same operators, quantization boundaries, residual scales, and width schedule. This correspondence is important: if the exported model differs semantically from the supernet path that won the search, final accuracy measures an implementation mismatch rather than architecture quality.

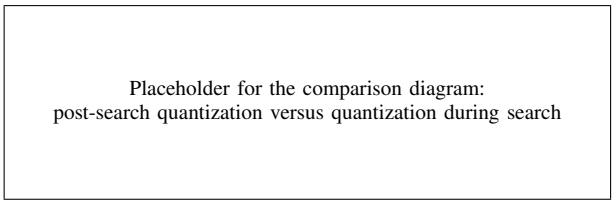
B. Quantized Mixed Operations

For convolution, the mutation hook obtains the effective sliced weight tensor associated with the current structural arguments. It applies a signed, per-output-channel LSQ quantizer and then evaluates the convolution. Grouped convolution is handled explicitly so that depthwise candidates remain valid when NNI represents their channel arguments inside a mixed operation. For linear layers, the same procedure is applied with a per-tensor LSQ scale.

The attention hook covers both the packed query/key/value projection used when all embedding dimensions match and the separate projection form used otherwise. It quantizes projection weights, input tokens, projected queries and keys, attention scores, softmax probabilities, the weighted context, and the output projection. Signed quantizers are used before and after linear projections and for scores; the softmax probability tensor uses an unsigned quantizer with a range-covering initialization. Masks are applied after score quantization so



(a) Quantization-aware training of one operation.



(b) Full-precision DARTS followed by QAT versus QA-DARTS.

Fig. 1. Conceptual bridge from QAT to QA-DARTS. The final diagrams will replace the framed placeholders while preserving the reserved layout area.

that a masked negative infinity is never rounded into a finite, attendable value.

Quantizers are registered as proper submodules before optimizer construction. This detail prevents a common implementation failure in which a learnable LSQ step size is created lazily during the first forward pass and is therefore absent from the optimizer. The code also verifies at training start that all mutable modules were replaced by the intended quantized runtime classes and that no standard mutable layer silently survived mutation.

C. Quantized Spatial Self-Attention

The attention candidate deserves an explicit formulation because a partially quantized implementation could receive an unfair advantage over convolutional candidates. Let $X \in \mathbb{R}^{N\times C}$ denote the $N = HW$ spatial tokens after an optional 1×1 input projection and layer normalization. For one attention head of dimension $d = C/h$, QA-DARTS computes

$$\widehat{X} = Q_s(X), \tag{6}$$

$$Q = Q_s(\widehat{X}\, Q_w(W_Q)), \quad K = Q_s(\widehat{X}\, Q_w(W_K)), \tag{7}$$

$$V = Q_s(\widehat{X}\, Q_w(W_V)), \tag{8}$$

where Q_w is signed LSQ for weights and Q_s is signed LSQ for activations. The score and probability paths are

$$S = Q_s\left(\frac{QK^{\mathsf{T}}}{\sqrt{d}}\right), \tag{9}$$

$$A = Q_u(\mathrm{softmax}(S + M)), \tag{10}$$

with mask M and unsigned probability quantizer Q_u. Finally,

$$Y = Q_s(Q_s(AV)\, Q_w(W_O)). \tag{11}$$

The heads are concatenated before the output projection, reshaped to NCHW format, batch-normalized, and passed through ReLU.

The ordering in (10) is important. Score quantization occurs before the mask is added. If a masked value of $-\infty$ were passed through a finite quantizer, clipping could turn it into an ordinary negative number and softmax would assign it nonzero probability. Applying the mask after quantization preserves exclusion semantics. Probability quantization occurs after softmax and optional dropout because those values occupy the bounded interval $[0, 1]$ and are statistically different from the signed scores.

The implementation uses eight heads and does not search the number of heads. Searching it as a differentiable scalar inside NNI's mixed multi-head operation would change tensor partitioning and could invalidate weight sharing. Treating the complete attention module as one LayerChoice candidate makes its topology discrete at the same level as a convolutional block. It also ensures that all candidates at a stage produce exactly the same output shape, a requirement for the weighted sum in (1).

Attention weights are per-tensor quantized, whereas convolutional filters are per-output-channel quantized. This choice reflects both implementation simplicity and the packed query/key/value representation used by PyTorch and NNI. Per-channel attention quantization is a possible extension, but it would require a carefully defined axis for the packed matrix and an equivalent frozen-model implementation. The present policy prioritizes exact agreement between the relaxed and exported networks.

The attention candidate includes an input projection only when input and output channels differ. When dimensions already match, the projection is the identity, preventing an unnecessary learned layer from biasing the comparison. Because L5–L7 operate at 8×8, the attention matrix for each image and head has only 64×64 entries. This restriction prevents the quadratic memory term from dominating the supernet, in which all candidates of a LayerChoice are evaluated during a differentiable forward pass.

D. Always-Residual Block

Every searchable stage is wrapped in an always-residual block. Let o_l denote the selected or relaxed main operation at layer l, P_l the identity or a 1×1 projection used to align channels, and c_l a learned residual multiplier. The block computes

$$x_{l+1} = \mathrm{ReLU}\left(Q_l^{+}\left(o_l(x_l)\right) + Q_l^{\pm}\left(c_l P_l(x_l)\right)\right), \tag{12}$$

where Q_l^{+} is an unsigned activation quantizer and $Q_l^{\pm}$ is signed when the projection ends in batch normalization. If P_l is an identity operating on a ReLU output, the two-bit traffic-sign implementation uses the full unsigned codebook on that branch.

Rather than fixing the residual strength manually for each experiment, the current model learns

$$c_l = 2\,\sigma(\rho_l), \tag{13}$$

which constrains c_l to $(0, 2)$ while retaining an unconstrained parameter ρ_l. The default initialization $\rho_l = 0$ gives $c_l = 1$. The two-bit traffic-sign configuration starts at $c_l = 0.75$ to preserve the stable operating point used by earlier low-bit experiments, while still permitting subsequent learning.

Separately quantizing the branches before addition is deliberate. Quantizing only the sum would let an unquantized residual bypass the target-precision constraint; quantizing the input once before both paths would force statistically different tensors to share a scale. Branch-local quantizers instead allow the learned ranges to adapt to the main and shortcut distributions.

E. Current Search Space

Table I lists only the searchable decisions used by the current code. The quantized stem, global average pooling, and quantized classifier are fixed components shown separately in Fig. 2. Seven residual stages and three downsampling locations are searchable. The number of discrete architectures is

$$2 \cdot 2 \cdot 4 \cdot 4 \cdot 3 \cdot 3 \cdot 3 \cdot 2^3 = 13{,}824. \tag{14}$$

This count excludes learned weights, LSQ parameters, and residual scales; it counts only exported structural choices.

The zero main branch in L3 or L4 does not remove the entire stage. It yields a residual-only block because the outer shortcut remains active, including a learned projection when channel dimensions change. Unlike a skip candidate placed directly in a DARTS mixture, the zero branch is evaluated within the same branch quantization and residual-fusion mechanism as every other option. Restricting it to two middle stages prevents the network from collapsing into a sequence of parameter-free shortcuts.

The dilated candidate expands context at a modest arithmetic cost. With dilation $d = 2$ and kernel width $k = 3$, the effective receptive-field width is

$$k_{\text{eff}} = k + (k - 1)(d - 1) = 5. \tag{15}$$

The physical operation still contains nine depthwise coefficients, preserving the 3×3 kernel constraint. The pointwise projection then performs channel mixing.

The attention candidate is intentionally late. If a feature map has $H \times W$ positions, the score tensor has complexity $O((HW)^2C)$. At 32×32, this term would be 256 times larger than at 8×8. Restricting attention to L5–L7 makes global context available while keeping memory and search-time cost controlled. Eight heads are used in every attention candidate; the channel counts 128, 256, and 200 are all divisible by eight.

F. Operation Cost and Hardware Interpretation

For a feature map of size $H \times W$, a dense $k \times k$ convolution with C_{in} input channels and C_{out} output channels requires approximately

$$HWk^2C_{\text{in}}C_{\text{out}} \tag{16}$$

multiply–accumulate operations. The depthwise-separable alternative requires

$$HW\left(k^2C_{\text{in}} + C_{\text{in}}C_{\text{out}}\right). \tag{17}$$

The zero main branch adds no main-path multiplication, although its residual projection can still incur a 1×1 cost when channels change. These alternatives let DARTS decide where dense feature transformation is justified and where feature reuse or factorized processing is sufficient.

The search is presently accuracy-driven rather than directly multi-objective. Equations (16) and (17) are therefore not added to the loss. Nevertheless, the discrete operator set exposes architectures with substantially different arithmetic, parameter, and anticipated optical resource demands. Recording these costs for every exported model enables a post-search Pareto analysis, and the same quantities can later be promoted to differentiable regularizers when hardware measurements become available.

G. Discretization and Deterministic Reconstruction

At the end of search, NNI exports the maximum-probability candidate at every LayerChoice. QA-DARTS stores semantic candidate names alongside architecture probabilities, avoiding the fragile practice of interpreting candidates solely by numeric index. The deterministic model reconstructs standard, depthwise, dilated, zero, and attention paths from the exported dictionary and retains the same width schedule, residual-scale parameterization, quantization boundaries, and downsampling choices.

All learned supernet weights are discarded for the primary evaluation. The reconstructed architecture is initialized again and optimized from scratch. This design distinguishes two questions: whether the relaxed supernet can fit the data and whether the exported discrete architecture is intrinsically trainable at the target precision. Reusing supernet weights would mix these questions and could favor operations that received more optimization during the shared-weight search.

V. Precision Management and Optimization

A. Weight and Activation Quantization Policy

The quantization policy follows the statistics of each tensor rather than applying one rule indiscriminately. Convolutional filters use signed per-output-channel LSQ because different filters may have markedly different ranges. Linear and attention matrices use signed per-tensor LSQ. Tensors following ReLU use unsigned activation LSQ and can devote all 2^b codes to non-negative values. Projection residuals and normalized attention tensors use signed activation LSQ. Softmax probabilities use unsigned quantization over $[0, 1]$.

The four- and two-bit activation quantizers perform a one-time data-dependent initialization on their first training batch. Given a clipped activation tensor a and $Q_p = 2^b - 1$, the initial step is proportional to

$$s_0 = \frac{2\,\mathbb{E}[|a|]}{\sqrt{Q_p}}. \tag{18}$$

Thereafter, the step and clipping bound are learned. The six-bit path starts from a fixed step and also learns it during training. For the multi-precision two-bit traffic-sign curriculum, independent LSQ states are maintained for two, four, and six bits so that returning to a precision does not overwrite another precision's calibrated range.

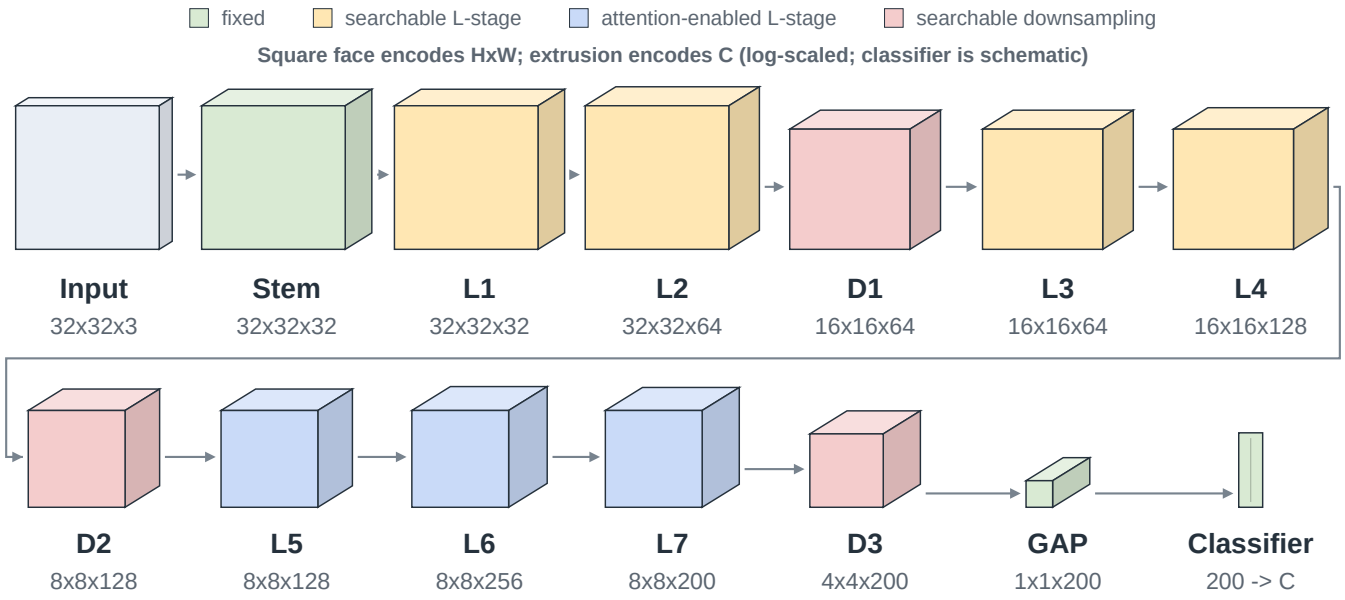


Fig. 2. Topology of the current QA-DARTS supernet. Seven components are shown per row; flow proceeds across the upper row and wraps from L4 to D2 on the lower row. For spatial tensors, the square front face represents $H \times W$ and the backward extrusion represents the channel count C; both are logarithmically scaled, while the final classifier is schematic. Labels report exact output shapes. Colors distinguish fixed components, searchable residual stages, attention-enabled stages, and downsampling decisions. L1–L7 are always-residual stages, and attention is restricted to the 8×8 stages.

TABLE I
SEARCHABLE DECISIONS IN THE CURRENT SEARCH SPACE

Search decision	Channels	Input size	Candidate operations	Choices
L1	$32 \rightarrow 32$	32×32	C3, DS3	2
L2	$32 \rightarrow 64$	32×32	C3, DS3	2
D1	$64 \rightarrow 64$	32×32	AP2, PC1	2
L3	$64 \rightarrow 64$	16×16	C3, DS3, Z, DDS3	4
L4	$64 \rightarrow 128$	16×16	C3, DS3, Z, DDS3	4
D2	$128 \rightarrow 128$	16×16	AP2, PC1	2
L5	$128 \rightarrow 128$	8×8	C3, DS3, A8	3
L6	$128 \rightarrow 256$	8×8	C3, DS3, A8	3
L7	$256 \rightarrow 200$	8×8	C3, DS3, A8	3
D3	$200 \rightarrow 200$	8×8	AP2, PC1	2

Operation legend: C3, standard 3×3 convolution; DS3, depthwise 3×3 plus pointwise 1×1 convolution; DDS3, dilation-two DS3; Z, zero main branch; A8, eight-head spatial self-attention; AP2, 2×2 average pooling; PC1, stride-two 1×1 convolution with batch normalization.

Two-bit width note: For the widened CIFAR-10 model, channels from the stem through L7 are $64, 64, 128, 128, 256, 256, 512, 200$; candidates and resolutions are unchanged.

B. Two-Bit Stabilization

Two-bit training is qualitatively harder than four- or six-bit training: weights have only four signed codes and unsigned activations only four non-negative codes. An abrupt transition from random initialization to two-bit computation can yield noisy gradients before useful features or quantizer ranges have formed. The current code uses two stabilization strategies, selected according to the dataset.

For CIFAR-10, the two-bit supernet is widened from the baseline channel schedule $[32, 32, 64, 64, 128, 128, 256, 200]$ to $[64, 64, 128, 128, 256, 256, 512, 200]$. The classifier input remains 200 to preserve the fan-in constraint. Search begins with 30 full-precision epochs; architecture updates are also delayed for those 30 epochs. Training then switches directly to two-bit computation, while validation always executes the target two-bit path. The deterministic two-bit model uses a shorter staged start: 10 full-precision epochs, 20 four-bit epochs, and then the target precision.

For the smaller traffic-sign dataset, the search follows a progressive quantization schedule: 50 epochs at six bits, 50 at four bits, 30 at two bits for adaptation, and the remainder at two bits. Architecture updates are held during the first 130 epochs. At each $6 \rightarrow 4$ or $4 \rightarrow 2$ transition, LSQ weight steps are refreshed in place from the active weights, optimizer moment state is cleared, and batch-normalization running statistics are reset. The parameter objects themselves are preserved so that the optimizer continues to track the correct trainable variables. The deterministic two-bit traffic-

Algorithm 1 QA-DARTS Search and Export

Define photonic-compatible ModelSpace $\mathcal{S}$
Mutate convolution, linear, and attention modules into LSQ mixed operations
Initialize weights w, architecture logits α, quantizer parameters s, and residual logits ρ
for epoch $t = 0, \ldots, T-1$ **do**
  Set active precision according to the experiment schedule
  for each training/validation mini-batch pair **do**
    Update w, s, ρ using quantized training loss
    if architecture warm-up is complete **then**
      Update α using quantized validation loss
    end if
  end for
  Record softmax probabilities, entropy, margins, and validation metrics
end for
Export $o_l^\star = \arg\max_o \alpha_{l,o}$ at every decision
Reconstruct deterministic model and reinitialize all trainable parameters
Train and evaluate the fixed architecture at the target bit width

sign model uses 50 full-precision and 50 four-bit epochs before continuing at two bits.

Validation is always performed at the final target precision, independently of the active warm-up phase. This avoids reporting optimistic validation accuracy from a higher-precision path and ensures that checkpoint selection is aligned with deployment conditions.

C. Optimization and Reproducibility

Network weights, quantizer parameters, and residual scales are optimized with stochastic gradient descent (SGD), momentum 0.9, and cosine annealing to a minimum learning rate of 10^{-3}. CIFAR-10 uses no weight decay in the current search code; GTSDB uses 5×10^{-4}. The architecture learning rate is 10^{-3}. Gradients are clipped to a maximum norm of one, which reduces instability from the STE and from low-bit precision transitions.

The implementation fixes the Python, NumPy, and PyTorch random seeds to 42, initializes CUDA seeds when available, enables deterministic PyTorch algorithms, disables cuDNN benchmarking, and disables tensor-float-32 paths. Data-loader workers receive deterministic seeds. These settings reduce run-to-run variation, although exact reproducibility can still depend on the software stack and hardware implementation.

Architecture logits are excluded from the ordinary SGD optimizer because NNI's DARTS strategy owns their update. Runtime checks count the installed quantized mixed operations and fail if any mutable convolution, linear, or attention module bypasses the custom hooks. Alpha trajectories are captured once per epoch and logged with semantic operation labels for probability, entropy, and polarization plots.

VI. Experimental Methodology

A. Research Questions

The updated experiments are designed to answer four questions without reusing numerical claims from the previous search space.

1) Can the current QA-DARTS search converge to well-polarized decisions at six, four, and two bits?
2) Which operation families are selected as precision decreases, especially dilated convolution, attention, and residual-only middle stages?
3) How much accuracy is lost between the relaxed supernet and the discretized architecture retrained from scratch?
4) What parameter, operation-count, and approximate arithmetic trade-offs arise from the discovered structures?

The primary outcome is test accuracy of the retrained discrete architecture. Search validation accuracy is diagnostic rather than final. Secondary outcomes include search entropy, top-one probability margin, parameter count, multiply–accumulate estimates, and wall-clock time.

B. Datasets and Partitions

CIFAR-10 contains 60,000 color images of size 32×32 in ten balanced classes [17]. The official 50,000-image training set is split stratifiably into 40,000 training and 10,000 validation samples. The official 10,000-image test set is held out until final evaluation. Training augmentation consists of a random 32×32 crop after four-pixel padding, horizontal flipping, rotation by at most 15°, and Cutout with a 16×16 mask. Validation and test images are converted to tensors without augmentation.

The German Traffic Sign Detection Benchmark (GTSDB) contains road-scene images and bounding-box annotations for traffic signs [18]. The current loader converts the detection benchmark into a 43-class classification task. Every annotated sign is cropped from its source image, resized to 32×32, and associated with its class label. Images numbered up to 599 define the training pool; later images form the held-out test pool. Twenty percent of the training crops from each available class are assigned to validation. Training augmentation uses random rotation by at most 15°, brightness/contrast/saturation/hue jitter, and Cutout. Horizontal flips are not used because a flip can change traffic-sign semantics. Validation and test crops receive only resizing and tensor conversion.

The training subset updates network and quantizer parameters, while the validation subset updates architecture parameters and supports checkpoint selection. The final test set is used only after the architecture has been exported and retrained from scratch.

C. Experiment Matrix

Table II records the configurations currently implemented. The six- and four-bit searches share the baseline backbone. Two-bit CIFAR-10 uses the widened channels described in Table I; two-bit GTSDB retains the baseline widths but uses the progressive precision curriculum.

TABLE II
CURRENT SEARCH AND RETRAINING PROTOCOL

Dataset / precision	Batch	Search epochs	Weight LR	Weight decay	Stabilization and retraining schedule
CIFAR-10 / 6 bit	128	400	2×10^{-2}	0	Direct six-bit search; deterministic model retrained for 400 epochs
CIFAR-10 / 4 bit	128	400	2×10^{-2}	0	Data-calibrated activation LSQ; deterministic model retrained for 400 epochs
CIFAR-10 / 2 bit	128	400	2×10^{-2}	0	Widened backbone; 30 full-precision search warm-up epochs; scratch schedule: 10 full precision, 20 four bit, then two bit
GTSDB / 6 bit	32	200	2×10^{-2}	5×10^{-4}	Direct six-bit search; deterministic model budget 250 epochs
GTSDB / 4 bit	32	200	2×10^{-2}	5×10^{-4}	Data-calibrated activation LSQ; deterministic model budget 250 epochs
GTSDB / 2 bit	32	400	5×10^{-2}	5×10^{-4}	Search: 50 epochs at six bit, 50 at four bit, 30 at two-bit adaptation, then two bit; scratch: 50 full precision, 50 four bit, then two bit

All searches use architecture learning rate 10^{-3}, gradient clipping at one, and cosine annealing of the network learning rate to 10^{-3}. Search-time drop path is disabled. The final deterministic model is initialized independently, trained with the exported operations, and evaluated with quantized inference.

D. Architecture-Search Diagnostics

An exported architecture alone does not reveal whether DARTS made confident decisions. For each decision l, the recorded softmax distribution $p_{l,k}$ is summarized by normalized entropy

$$\tilde{H}_l = -\frac{1}{\log K_l} \sum_{k=1}^{K_l} p_{l,k} \log p_{l,k}, \tag{19}$$

which ranges from zero for a one-hot distribution to one for a uniform distribution. A complementary margin is

$$m_l = p_{l,(1)} - p_{l,(2)}, \tag{20}$$

where $p_{l,(1)}$ and $p_{l,(2)}$ are the largest and second-largest probabilities. Low entropy and a large margin indicate a robust argmax; high entropy or a small margin signals that the discrete choice may be sensitive to random seed or optimization noise.

The final report will plot trajectories of $p_{l,k}$, $\tilde{H}_l$, and m_l for all 13 decisions. Operation names are preserved semantically, so plots will identify, for example, *dilated depthwise* or *attention* rather than an implementation-dependent option index.

E. Final-Model Evaluation

Each exported model will be retrained with at least the same seed and data partition used in search. Final reporting will include best validation accuracy, corresponding held-out test accuracy, parameter count, estimated multiply–accumulates, selected operation sequence, and training time. To separate architectural effects from optimization variance, repeated scratch runs should be reported with mean and standard deviation once compute permits. Confusion matrices and per-class recall are particularly relevant for GTSDB because the cropped class distribution is not balanced.

The comparison between supernet and scratch training will use curves on the same epoch axis but will not interpret supernet accuracy as deployable-model accuracy. A supernet layer evaluates a weighted sum of several candidates, whereas the exported model retains only one. The difference between the two curves therefore measures both discretization loss and the consequences of shared-weight training.

F. Ablations

Three ablations are planned to attribute gains to individual changes in the current implementation. First, the new search space will be compared with a convolution-only space that removes attention, dilation, and the zero main branch while preserving widths and quantization. Second, the learned residual scale in (13) will be compared with a fixed unit scale. Third, the two-bit curricula will be compared with direct two-bit training. Each ablation must retain the same data split, target precision, optimizer, and retraining budget so that the changed component is the only systematic difference.

For attention-containing architectures, a further diagnostic will compare quantized and unquantized attention internals while keeping convolutional quantization active. This is not a deployment configuration; it tests whether any observed preference for attention is caused by its representational value or by an accidental precision bypass.

VII. RESULTS

VIII. Limitations and Future Directions

The framework models a constrained photonic target but does not yet simulate a specific fabricated device. The six-bit ceiling, non-negative activation path, fan-in limit, and kernel restrictions are architectural proxies. Thermal drift, shot noise, component mismatch, optical loss, calibration frequency, and converter energy are not injected into the current forward model. Conclusions about physical deployment will therefore require hardware measurements or a validated device simulator.

Search cost is also not an explicit objective. The operator set contains efficient alternatives, but DARTS minimizes validation loss alone. A natural extension is multi-objective search with differentiable latency, energy, optical component count, or conversion penalties. Such costs should be calibrated on the intended platform; generic multiply–accumulate counts do not capture wavelength routing, electro-optic conversion, memory traffic, or attention-specific data movement.

The search uses one fixed precision per final model. Mixed-precision search could allocate more bits to sensitive projections and fewer bits to robust depthwise or residual paths. The multi-precision quantizer state already implemented for the two-bit curriculum provides part of the required infrastructure, but bit width would need to become a properly regularized architecture decision rather than a training-only schedule.

The evaluation is limited to two 32×32 image-classification tasks. Larger images and non-vision workloads may change the value of attention, dilation, and downsampling. Time-series inference, optical communications, and scientific signal processing are especially relevant future domains. Finally, physical validation on a PNN prototype is essential. It would determine whether the architecture preferences learned from quantization and structural constraints survive real optical noise and calibration behavior.

IX. Conclusion

This article formulated QA-DARTS as an end-to-end method for searching neural architectures under photonic-compatible precision and structural constraints. The updated framework quantizes the relaxed supernet rather than applying low precision after architecture selection, and it maintains semantic equivalence between search and deterministic retraining. Its current search space combines seven always-residual stages, learned residual scaling, efficient and dilated convolutions, residual-only middle blocks, late spatial self-attention, and searchable downsampling while preserving a 200-feature classification fan-in. Precision-specific warm-up schedules address the instability of two-bit optimization without changing the target-precision validation path. The resulting methodology provides a controlled basis for evaluating which architectural patterns remain useful as numerical resolution decreases. Quantitative conclusions will be added after the updated searches, scratch retraining runs, and plots have been completed.

Acknowledgments

The author thanks Marco Cococcioni, Nicola Andriolli, and Emilio Paolini for supervision and guidance on photonic-aware neural-network design and quantization-aware architecture search.

References

[1] L. De Marinis, M. Cococcioni, P. Castoldi, and N. Andriolli, "Photonic neural networks: A survey," *IEEE Access*, vol. 7, pp. 175827–175841, 2019.

[2] E. Paolini, L. De Marinis, M. Cococcioni, L. Valcarenghi, L. Maggiani, and N. Andriolli, "Photonic-aware neural networks," *Neural Computing and Applications*, 2022.

[3] Y. Shen, N. C. Harris, S. Skirlo, *et al.*, "Deep learning with coherent nanophotonic circuits," *Nature Photonics*, vol. 11, no. 7, pp. 441–446, 2017.

[4] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *Journal of Machine Learning Research*, vol. 20, no. 55, pp. 1–21, 2019.

[5] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[6] Microsoft, "Neural Network Intelligence (NNI): An open source AutoML toolkit," 2018. [Online]. Available: https://nni.readthedocs.io/en/stable/

[7] G. Bender, P.-J. Kindermans, B. Zoph, V. Vasudevan, and Q. V. Le, "Understanding and simplifying one-shot architecture search," in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 550–559, 2018.

[8] Z. Guo, X. Zhang, H. Mu, W. Heng, Z. Liu, Y. Wei, and J. Sun, "Single path one-shot neural architecture search with uniform sampling," in *Proc. Eur. Conf. Computer Vision (ECCV)*, pp. 544–560, 2020.

[9] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients," *arXiv:1606.06160*, 2018.

[10] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[11] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv:1308.3432*, 2013.

[12] P. Yin, J. Lyu, S. Zhang, S. Osher, Y. Qi, and J. Xin, "Understanding straight-through estimator in training activation quantized neural nets," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[13] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, "PACT: Parameterized clipping activation for quantized neural networks," *arXiv:1805.06085*, 2018.

[14] A. G. Howard, M. Zhu, B. Chen, *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv:1704.04861*, 2017.

[15] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[16] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[17] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.

[18] S. Houben, J. Stallkamp, J. Salmen, M. Schlipsing, and C. Igel, "Detection of traffic signs in real-world images: The German Traffic Sign Detection Benchmark," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2013.